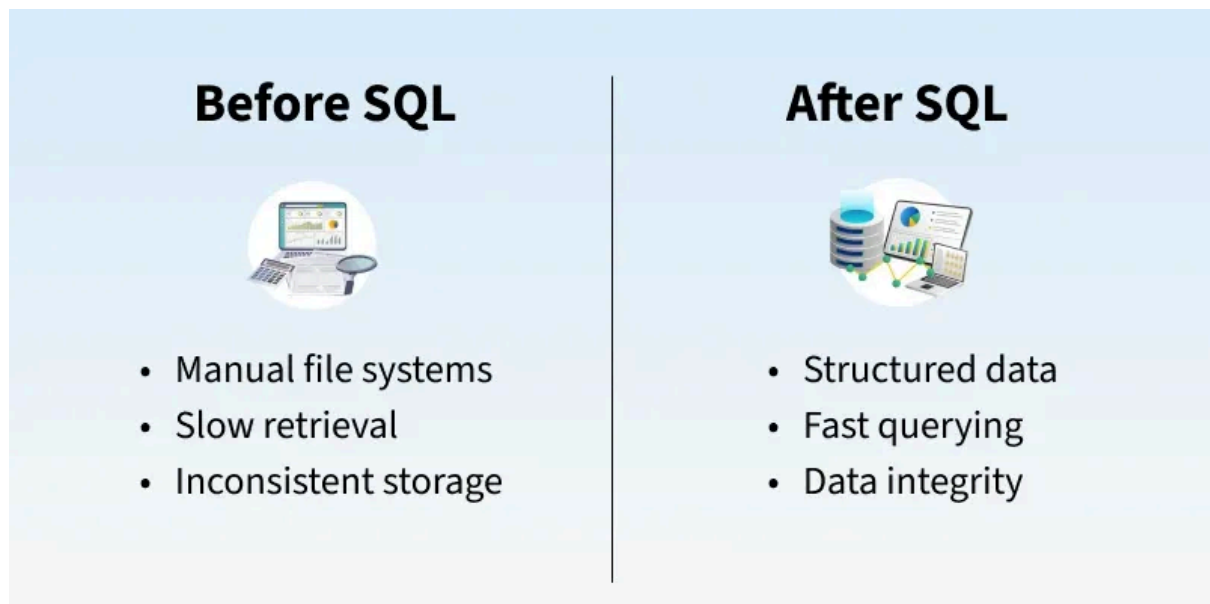


1. Introduction to Advanced SQL

Advanced SQL is an extension of basic SQL, used to handle complex data processing tasks in real-world systems. While basic SQL is primarily used for retrieving simple data from one or a few tables, Advanced SQL allows working with large datasets, complex structures, multi-layered relationships, and high-performance requirements.

Simply put, Advanced SQL not only answers the question "what is data?" but also helps answer "how is data organized, analyzed, and optimized?".

In practice, without using advanced techniques, the system will encounter problems such as slow queries, high resource consumption, and difficulty scaling as data increases.



2. Advanced Query Techniques

A subquery is a query placed inside another query. When SQL executes, the subquery runs first and returns the results for the outer query to use. Subqueries are often used when you need an intermediate value, such as a mean, a maximum value, or a data set to filter.

```
SELECT name  
FROM  
employees  
WHERE salary > (
```

```
SELECT AVG(salary) FROM employees  
);
```

The important point to understand is that there is a type of subquery called a correlated subquery, meaning it depends on each line of the outer query. This causes it to run repeatedly, leading to low performance if the data is large.

```
SELECT e1.name  
FROM employees  
e1 WHERE salary  
> (  
    SELECT AVG(salary)  
    FROM employees e2  
    WHERE e2.department = e1.department  
);
```

CTE stands for Common Table Expression, a temporary table created within the scope of an SQL statement. It doesn't exist in the database but only while the query is running. CTEs help break down complex logic into more readable parts, much like naming an intermediate result for reuse.

```
WITH AvgSalary AS (  
    SELECT department, AVG(salary) AS  
    avg_sal FROM employees  
    GROUP BY department  
)  
SELECT e.name,  
e.salary FROM  
employees e  
JOIN AvgSalary a ON e.department =  
a.department WHERE e.salary > a.avg_sal;
```

The diagram shows a SQL query with several handwritten annotations in blue ink:

- WITH keyword**: Points to the `WITH` keyword at the start of the CTE definition.
- CTE Name**: Points to `avg_price_brand` in the `WITH avg_price_brand AS` line.
- CTE body**: A dashed red box encloses the subquery `(SELECT brand, AVG(price) AS average_for_brand FROM cameras GROUP BY brand)`.
- Use of CTE**: Points to the `avg` alias in the `JOIN avg_price_brand avg` line.

```

WITH avg_price_brand AS
    (SELECT brand, AVG(price) AS average_for_brand
     FROM cameras
     GROUP BY brand)
SELECT c.id, c.brand, c.model, c.price, avg.average_for_brand
FROM cameras c
JOIN avg_price_brand avg
ON c.brand = avg.brand;

```

Recursive CTEs are a special type of CTE that allows querying hierarchically structured data such as directory trees or product catalogs. They work by starting from an initial dataset and then repeatedly appending subdata based on previous results. A stopping condition is crucial; otherwise, the query will run indefinitely.

```

WITH RecursiveCTE AS (
    SELECT id, parent_id,
           name FROM categories
    WHERE parent_id IS NULL

    UNION ALL

    SELECT c.id, c.parent_id, c.name
    FROM categories c
    JOIN RecursiveCTE r ON c.parent_id = r.id
)
SELECT * FROM RecursiveCTE;

```

3. Window Functions

A window function is a type of function that allows calculations to be performed on a group of related data rows without losing each individual row. This differs from a `GROUP BY` function, which combines multiple rows into a single row, while a window function preserves all rows but adds calculation information.

Window functions work based on the OVER clause, where you can divide data into groups using PARTITION BY and determine the order using ORDER BY. Each row is then calculated based on its associated data "window".

For example, if you want to rank employees by salary within each department, you can divide by department and sort by salary. Each employee will still be in a separate row, but with an added rank column.

```
SELECT department, name, salary,  
       ROW_NUMBER() OVER (PARTITION BY department ORDER BY  
salary DESC) AS row_num,  
       RANK() OVER (PARTITION BY department ORDER BY salary  
DESC) AS rank,  
       DENSE_RANK() OVER (PARTITION BY department ORDER BY  
salary DESC) AS dense_rank  
FROM employees;
```

4. Joins

Joining is a method of combining data from multiple tables based on the relationships between them. In relational databases, data is often split into multiple tables to avoid duplication, so joining is a necessary technique to combine data when querying.

-- INNER JOIN

```
SELECT e.name, d.department_name  
FROM employees e  
INNER JOIN departments d ON e.department_id = d.id;
```

-- LEFT JOIN

```
SELECT e.name, d.department_name  
FROM employees e  
LEFT JOIN departments d ON e.department_id = d.id;
```

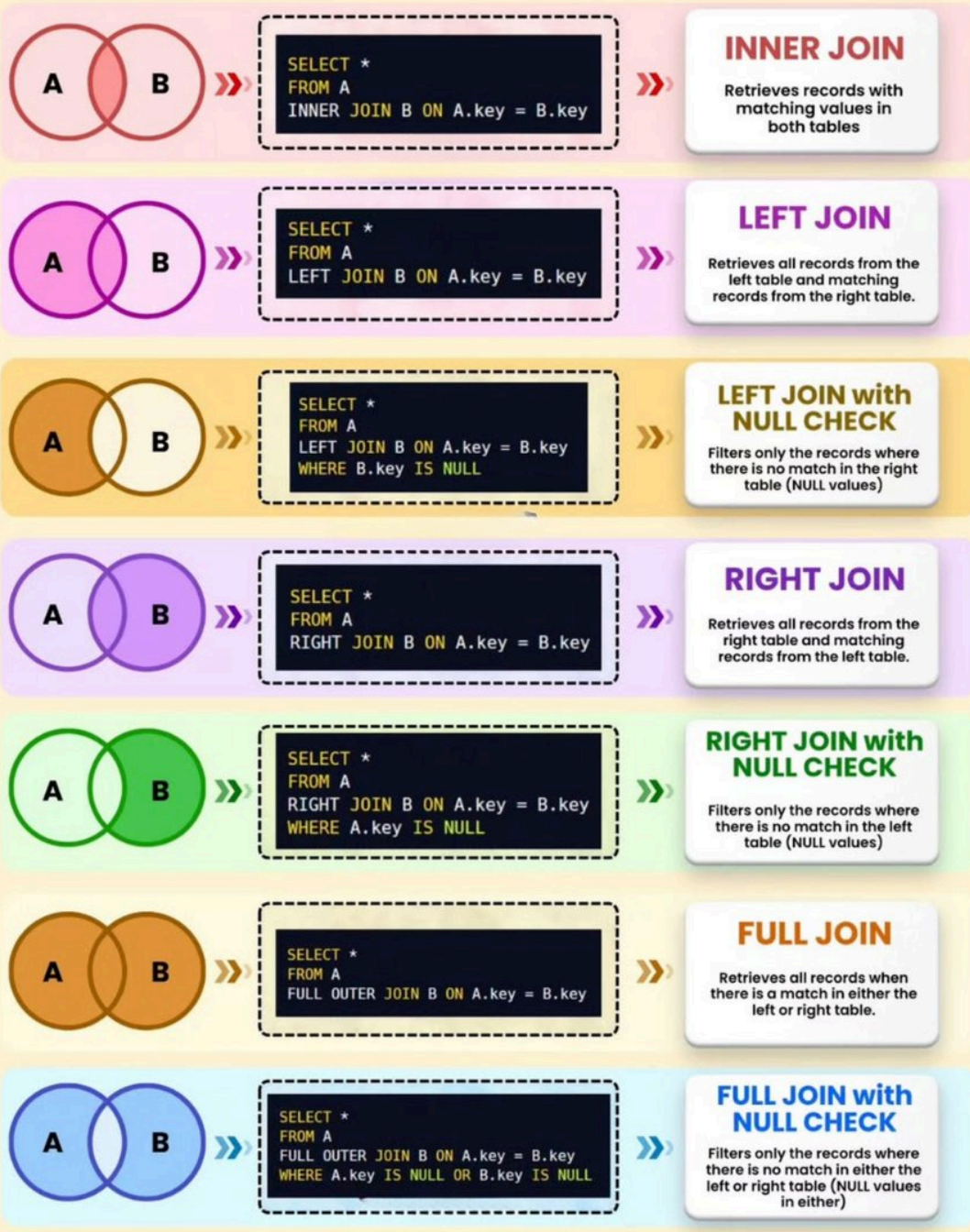
-- FULL JOIN

```
SELECT e.name, d.department_name  
FROM employees e  
FULL JOIN departments d ON e.department_id = d.id;
```

A self-join occurs when a table is joined to itself. This is often used to represent hierarchical relationships; for example, an employee might have a manager in the same table.

```
SELECT e1.name AS employee, e2.name AS  
manager FROM employees e1  
LEFT JOIN employees e2 ON e1.manager_id = e2.id;
```

SQL Joins CheatSheet



5. Indexing

An index is a data structure created to help databases search data faster. It works similarly to the table of contents in a book, allowing you to find the relevant page without having to read the entire book.

```
CREATE INDEX idx_salary ON employees(salary);
```

When there is no index:

```
SELECT * FROM employees WHERE salary > 5000;
```

The database will scan the entire table.

6. Query Optimization

Query optimization is the process of writing SQL statements so that the database executes as quickly as possible while using the fewest resources.

The execution plan is the plan for executing the database:

```
EXPLAIN SELECT * FROM employees WHERE salary > 5000;
```

Optimal example:

Not optimal:

```
SELECT * FROM employees;
```

Optimization:

```
SELECT name, salary  
FROM employees  
WHERE salary > 5000;
```

7. Transactions

A transaction is a set of database operations performed as a single unit.

```
BEGIN;
```

```
UPDATE accounts SET balance = balance - 100 WHERE id =  
1; UPDATE accounts SET balance = balance + 100 WHERE id  
= 2;
```

```
COMMIT;
```

Rollback:

```
ROLLBACK;
```

8. Concurrency Control

Concurrency control is a mechanism that allows multiple users or processes to access and modify data simultaneously without causing errors.

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Example lock:

```
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;
```

9. Stored Procedures and Functions

A stored procedure is a set of SQL statements that are pre-stored in a database.

```
CREATE PROCEDURE GetEmployees()  
BEGIN  
    SELECT * FROM employees;  
END;
```

Function:

```
CREATE FUNCTION GetBonus(salary INT)  
RETURNS INT  
BEGIN
```

```
    RETURN salary *  
    0.1; END;
```

10. Triggers

A trigger is a piece of code that automatically runs when an event occurs in the database.

```
CREATE TRIGGER before_insert_employee  
BEFORE INSERT ON employees  
FOR EACH ROW  
SET NEW.created_at = NOW();
```

11. Views

A view is a virtual table created from an SQL query.

```
CREATE VIEW high_salary_employees AS  
SELECT name, salary  
FROM employees  
WHERE salary > 5000;
```

Materialized view (PostgreSQL):

```
CREATE MATERIALIZED VIEW mv_employees  
AS SELECT * FROM employees;
```

12. Data Integrity and Constraints

```
CREATE TABLE  
    users ( id INT  
    PRIMARY KEY,
```

```
email VARCHAR(255)
UNIQUE, age INT CHECK
(age >= 18),
department_id INT,
FOREIGN KEY (department_id) REFERENCES departments(id)
);
```

13. Advanced Data Types

JSON:

```
SELECT data->>'name' FROM users;
```

Array:

```
SELECT * FROM users WHERE tags @> ARRAY['admin'];
```

14. Best Practices

-- Create a suitable index

```
CREATE INDEX idx_department ON employees(department_id);
```

Use LIMIT to reduce data usage.

```
SELECT * FROM employees LIMIT 10;
```

-- Monitoring slow queries (MySQL)

```
SHOW PROCESSLIST;
```